

OBJECT-ORIENTED PROGRAMMING (OOP)

Theory Question Paper

Time: 2 Hours	Maximum Marks: 50
Name: _____	Roll No.: _____

General Instructions:

1. Attempt all questions as directed in each section.
2. Marks are shown against each question.
3. Draw neat diagrams and give suitable examples wherever required.
4. Write answers in clear, well-organised points.

SECTION A: Very Short Answer Questions (5 x 2 = 10 Marks)

Attempt ALL questions. Answer each in 2-3 lines.

1. Define Object-Oriented Programming and list its main features. [2]
2. What is a destructor? When is it called? [2]
3. What is the role of the `final` keyword in Java? [2]
4. What is dynamic (late) binding? [2]
5. Define message passing in object-oriented programming. [2]

SECTION B: Short Answer Questions (5 x 4 = 20 Marks)

Attempt any FIVE of the following six questions.

6. Differentiate between a class and an object with a suitable example. [4]
7. What is a constructor? Explain the different types of constructors. [4]
8. Explain the difference between method overloading and method overriding. [4]
9. What is abstraction? How does it differ from encapsulation? [4]
10. Explain the `static` keyword and where it can be used. [4]
11. What is the difference between an abstract class and an interface? [4]

SECTION C: Long Answer Questions (2 x 10 = 20 Marks)

Attempt any TWO of the following four questions.

12. (a) Explain the four pillars of OOP with a real-world example for each. [6]
(b) List any four advantages of OOP over procedural programming. [4]
13. (a) Explain the different types of inheritance with neat diagrams. [6]
(b) Why does the diamond problem occur, and how do C++ and Java handle it? [4]
14. (a) What is polymorphism? Explain its types with suitable examples. [6]
(b) Differentiate between compile-time (static) and runtime (dynamic) polymorphism. [4]
15. (a) Explain the access specifiers (`public`, `private`, `protected` and `default`) and their scope, using a comparison table. [6]
(b) Explain the purpose of the `this` and `super` keywords with examples. [4]

*** END OF QUESTION PAPER ***

MARKING GUIDE (For Teacher Use Only)

Section A: Key Points Expected (2 marks each)

- Q1.** OOP is a paradigm that organises software around objects (data + behaviour) rather than functions. Features: encapsulation, abstraction, inheritance, polymorphism. (1 + 1)
- Q2.** A special method that cleans up an object's resources. Called automatically when the object is destroyed or goes out of scope (C++). Java uses garbage collection / finalizers. (1 + 1)
- Q3.** final variable = constant, final method = cannot be overridden, final class = cannot be inherited. (any 2 uses)
- Q4.** Method call is resolved at runtime based on the actual object type, not the reference type. Achieved via overriding / virtual functions. (1 + 1)
- Q5.** Objects communicate by sending messages, i.e. calling each other's methods with arguments. (2)

Section B: Key Points Expected (4 marks each)

- Q6.** Class = blueprint/template (no memory allocated); object = instance of a class (memory allocated). Example: class Car, object myCar. (2 for definitions, 2 for example)
- Q7.** Special method with the class name and no return type, called automatically on object creation. Types: default, parameterized, copy. (1 definition, 3 for types)
- Q8.** Overloading: same name, different parameters, same class, compile-time. Overriding: same signature in subclass, runtime. (any 4 valid differences)
- Q9.** Abstraction hides implementation and shows only essential features (design level). Encapsulation wraps data and methods and restricts access (implementation level). (2 + 2)
- Q10.** Static variable, method, block, nested class. Belongs to the class, not an object; shared by all instances; accessed without creating an object. (2 for concept, 2 for uses)
- Q11.** Abstract class: can have concrete methods and state, single inheritance. Interface: contract of behaviour, multiple implementation allowed. (any 4 points)

Section C: Marking Scheme (10 marks each)

- Q12.** (a) Encapsulation, abstraction, inheritance, polymorphism: 1 mark for definition + 0.5 mark for example each. (b) Any four advantages (reusability, modularity, security, easier maintenance, etc.): 1 mark each.
- Q13.** (a) Single, multilevel, hierarchical, multiple, hybrid: 1 mark each, diagram credit included; 1 mark for overall presentation. (b) Diamond problem arises when a class inherits the same base class through two paths, causing ambiguity (2). C++ uses virtual inheritance; Java disallows multiple class inheritance, and interface default-method conflicts must be resolved explicitly (2).
- Q14.** (a) Definition (1); compile-time polymorphism with example (2.5); runtime polymorphism with example (2.5). (b) Any four differences: binding time, mechanism (overloading vs overriding), speed, flexibility, inheritance requirement. 1 mark each.
- Q15.** (a) Correct scope of each specifier: 1 mark each (4), plus comparison table (2). Table columns: same class, same package, subclass, outside package. (b) *this*: refers to current object, resolves name clashes, calls another constructor (2). *super*: refers to parent class, calls parent constructor or overridden method (2).

Note: Award marks for alternative correct answers and relevant examples. Give partial credit for incomplete but correct points.